



Soutenance intermédiaire

Team Prague

OCR Sudoku Solver

Simon BOUSQUET
Oleh KHYTREYKO
Léo HABETS
Nicolas LEFEVRE-PONTALIS

Novembre 2023

Table des matières

1	Introduction	3
1.1	La Team Prague	3
1.1.1	Simon BOUSQUET (Chef de Projet)	3
1.1.2	Nicolas LEFEVRE-PONTALIS	3
1.1.3	Léo HABETS	3
1.1.4	Oleh KHYTREYKO	4
1.2	Répartition des tâches	4
1.3	Avancement des tâches	4
2	Prétraitement de l'image	5
2.1	Filtres	5
2.1.1	Grayscale	5
2.1.2	Contraste	5
2.1.3	Médian	5
2.1.4	Thresholding : Filtre de seuillage	6
2.1.5	Inversion	6
2.2	Resultat Final	6
2.3	Rotation manuelle de l'image	7
2.3.1	Introduction	7
2.3.2	Processus de rotation	7
2.3.3	Résultats	8
2.4	Redimension manuelle de l'image	9
2.4.1	Details	9
2.4.2	Résultats	9
3	Segmentation de l'image	10
3.1	Transformée de Hough	10
3.1.1	Principe	10
3.2	Découpage de l'image	13
4	Réseau de neurones	14
4.1	Fonctionnement du réseau de neurones	14
4.2	Résultats	14
5	Résolveur de sudoku	15
6	Interface utilisateur	16
7	Conclusion	17

1 Introduction

Ce rapport offre une synthèse de l'état d'avancement du projet mené par la Team Prague au cours de la période de travail préalable à la première soutenance. Il aborde en détail de nombreux aspects, notamment la réalisation des différentes fonctionnalités et les difficultés rencontrées, en mettant en lumière les méthodes employées qui seront détaillées dans les sections correspondantes.

1.1 La Team Prague

1.1.1 Simon BOUSQUET (Chef de Projet)

Ma première expérience en traitement d'image en utilisant un langage de programmation remonte à 2018, durant un atelier de programmation organisé par mon collège. Nous avions pour but de modifier certaines données d'une image telle que sa couleur dominante, sa taille, etc... En dehors de cela, je n'ai qu'à quelques occasions eu à réaliser ce genre de tâches, principalement durant les TPs de C et de C# à l'Epita. J'appréhende la difficulté de certaines opérations telles que la rotation d'image en raison du caractère exigeant du langage C, que ce soit vis à vis de la manipulation de mémoire ou plus globalement des calculs mathématiques, mais je reste confiant et surtout déterminé à mener à bout ce projet.

1.1.2 Nicolas LEFEVRE-PONTALIS

Je suis passionné d'informatique depuis mes 13 ans. J'ai touché à beaucoup de chose dans ce domaine et entrepris plusieurs projets dans divers langages de programmation. Ayant réalisé la grande majorité de ces projets en solo (hormis le projet de S2), cette OCR va me permettre de beaucoup développer mes compétences de travail en équipe. J'ai surtout très hâte de travailler autour des réseaux de neurones et de la détection de chiffre car c'est un domaine qui m'intéresse beaucoup depuis des années. C'est avec beaucoup de motivation et d'enthousiasme que je me lance dans ce projet.

1.1.3 Léo HABETS

Dès mes années lycée, j'ai eu mon premier contact avec la programmation, en explorant de nombreuses vidéos de divertissement. En particulier, j'ai été impressionné par celles abordant le langage Brainfuck. En classe de seconde, lorsque j'ai découvert Python, et grâce à un ami en STI2D, j'ai commencé à m'amuser à créer de petits programmes, en mettant l'accent sur le développement d'applications mathématiques pratiques. J'ai rapidement constaté que l'aspect mathématique des algorithmes, notamment ceux liés au traitement d'images et à la détection de formes géométriques, constituait une source d'intérêt majeur au cours de la première phase d'implémentation dans le projet du semestre 3.

1.1.4 Oleh KHYTREYKO

Passionné d'informatique depuis petit, j'ai eu l'occasion d'être confronté à plusieurs projets en solo comme en groupe. Ayant acquis pas mal d'expérience grâce au projet de s2, j'ai compris à quel point la communication dans un groupe peut être compliquée et comment une bonne organisation permet de simplifier beaucoup le travail. Travailler avec de l'imagerie et en particulier des filtres m'a permis de beaucoup apprendre dans ce domaine que je ne connaissais presque pas avant. J'ai hâte de relever ce nouveau défi et voir jusqu'où on pourra mener ce projet.

1.2 Répartition des tâches

Voici un tableau de la répartition des tâches pour cette première période de travail :

	Léo	Nicolas	Oleh	Simon
Gestion de l'image				
Prétraitement			Principal	
Rotation				Principal
Transformée de Hough	Principal			
Détection de la grille	Principal			
Découpage de l'image		Principal		
Reconnaissance				
Réseau de Neurones		Principal		
Sudoku				
Solver de sudoku		Principal		

1.3 Avancement des tâches

Voici un tableau de l'avancement des tâches pour cette première soutenance. Comme vous pouvez le voir, nous n'avons aucun retard.

Tâches	Prévu	Réalisé
Prétraitement	20%	80%
Rotation	50%	50%
Transformée de Hough	100%	100%
Détection de la grille	100%	100%
Découpage de l'image	80%	80%
Réseau de neurones XOR	100%	100%
Solver de sudoku	100%	100%

TABLE 1 – Répartition des tâches

2 Prétraitement de l'image

2.1 Filtres

Avant de se lancer dans la détection et la résolution de notre sudoku, un prétraitement de l'image convenable est primordiale. Pour se faire nous avons transformé notre image grâce à plusieurs filtres dans un ordre précis :

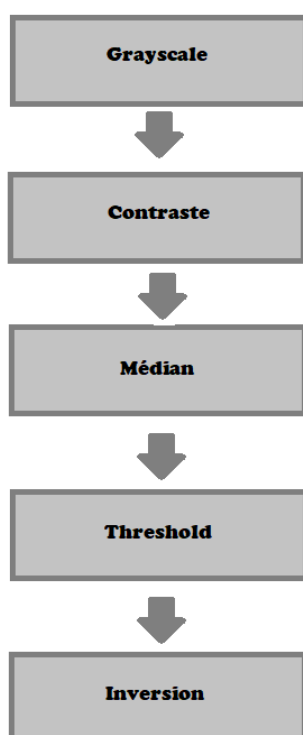


FIGURE 1 – Etapes du traitement

2.1.1 Grayscale

Le grayscale est un filtre permettant d'afficher une image en niveau de gris. On l'obtient en multipliant chaque composante RGB de notre pixels par des valeurs prédéfinies qui spécifient la profondeur du niveau de gris.

2.1.2 Contraste

Pour avoir l'image la plus optimale, il faut augmenter le contraste de celle-ci. Pour le faire, nous avons commencé par créer un histogramme de la valeur des pixels par rapport à leur intensité dans l'image. Pour finir, nous accentuons les pixels les plus présents.

2.1.3 Médian

Le filtre médian permet de donner un côté propre à l'image et d'enlever les impuretés (bruit) de type sel (grains blancs) et poivre (grains noirs). Ce filtre se base sur le principe suivant :

Nous commençons par prendre tous les pixels dans un carré de 3x3 autour de notre pixel de base. Ensuite nous rangeons chaque valeur et nous prenons la valeur médiane de cette liste pour la remplacer dans le pixel du milieu. Voici une petite illustration pour mieux visualiser ce filtre :

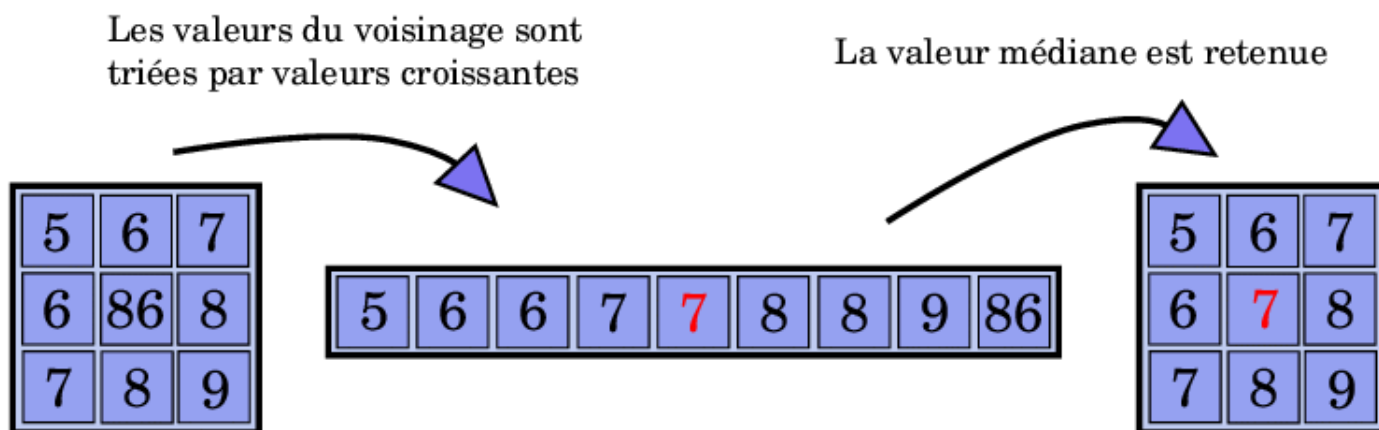


FIGURE 2 – Filtre médian

2.1.4 Thresholding : Filtre de seuillage

Ce filtre permet de passer à une image complètement en noir et blanc. Pour optimiser au max ce filtre, on regarde les valeurs de pixels de l'image et on adapte un seuil en fonction de celui-ci. Ce seuil nous servira pour déterminer si le pixels en question sera blanc si la valeur dépasse le seuil ou noir si celle-ci est inférieure au seuil.

2.1.5 Inversion

Ce dernier filtre n'a rien de particulier et ne fais qu'inverser les couleur blanc et noir pour permettre d'être en corrélation avec la suite...

2.2 Resultat Final

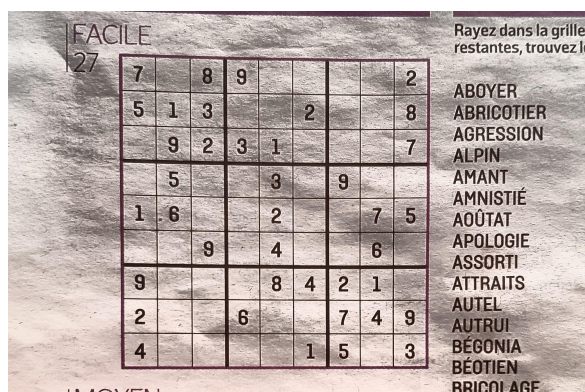


FIGURE 3 – Image originale

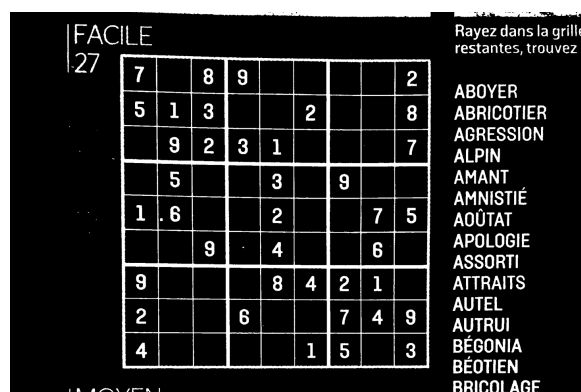


FIGURE 4 – Image finale

2.3 Rotation manuelle de l'image

2.3.1 Introduction

Une fois les premières étapes du traitement de l'image réalisés, il sera nécessaire de modifier le pivot de l'image afin de permettre une meilleure détection des cases du sudoku.

Nous avons donc pour cela implémenté un exécutable `./rotate {image angle}` où *image* est l'image à modifier et *angle* l'angle de rotation souhaité. Une fois l'exécution terminée, le fichier `.png` généré est stocké sous le nom `rotated.png`. Cet algorithme a été réalisé en utilisant les bibliothèques SDL2 et SDL_Image, et en se basant sur le principe de la matrice de rotation :

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

Dans l'exemple suivant, nous effectuons successivement une rotation de 45 degrés puis de -45 degrés sur l'image *Image originale*.

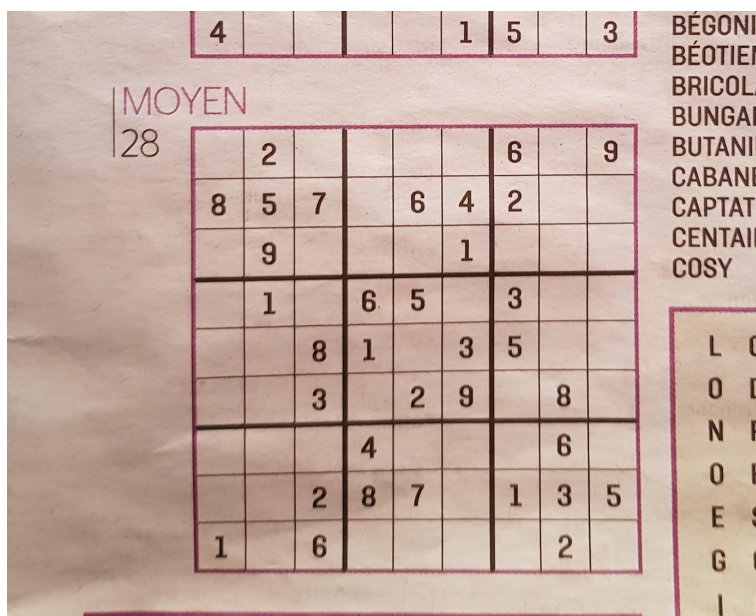


FIGURE 5 – Image originale

2.3.2 Processus de rotation

Premièrement, l'image est chargée, et un premier traitement lui est appliqué. Ainsi, si le format de l'image passée en paramètre est de 1 octet par pixel, cette dernière est convertie et renvoyée en format 3 octets par pixel afin de favoriser la réutilisabilité de l'image, et de diminuer la perte de qualité, au détriment d'une taille de fichier supérieure mais restant tout à fait acceptable. Une fois cette étape réalisée, l'image peut être pivotée.

Une nouvelle surface de largeur $newWidth$ et de hauteur $newHeight$, correspondant à la taille maximale nécessaire à l'inclusion de tous les pixels de l'image originale, est ensuite créée en suivant les formules suivantes, où $height$ et $width$ correspondent respectivement à la hauteur et la largeur de l'image originale :

$$newWidth = abs(width * \cos(\theta) + height * \sin(\theta))$$

$$newHeight = abs(width * \sin(\theta) + height * \cos(\theta))$$

Pour chaque pixel de la nouvelle surface (x,y), les coordonnées du pixel de la surface originale à placer à ces coordonnées sont ensuite calculés avec les formules suivantes :

$$OriginalX = ((x - newWidth/2) * \cos(\theta) - (y - newHeight/2) * \sin(\theta) + width/2)$$

$$OriginalY = ((x - newWidth/2) * \sin(\theta) + (y - newHeight/2) * \cos(\theta) + height/2)$$

Si les coordonnées n'existent pas sur la surface originale, un pixel blanc (0xFFFFFFFF) est ajouté à ces coordonnées, de manière à créer un arrière plan entièrement blanc.

Une fois chaque pixel placé, l'image est redimensionnée de façon à ce que les 4 coins visibles de l'image originale correspondent aux limites de la nouvelle image générée, et qu'aucun espace ne soit ainsi "gâché", ce qui réduirait la résolution de l'image finale, et limiterait la réutilisabilité de l'image.

2.3.3 Résultats

De cette façon, il devient possible de manipuler la même image dans le sens inverse, afin de récupérer l'image originale.

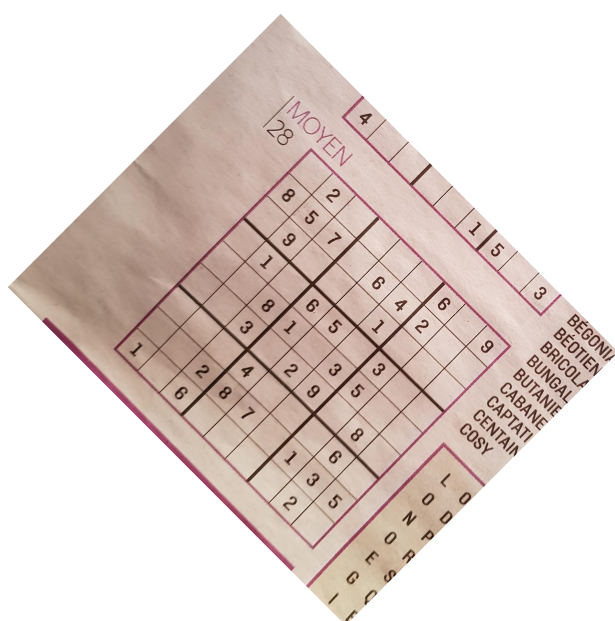


FIGURE 6 – Rotation de 45 degrés

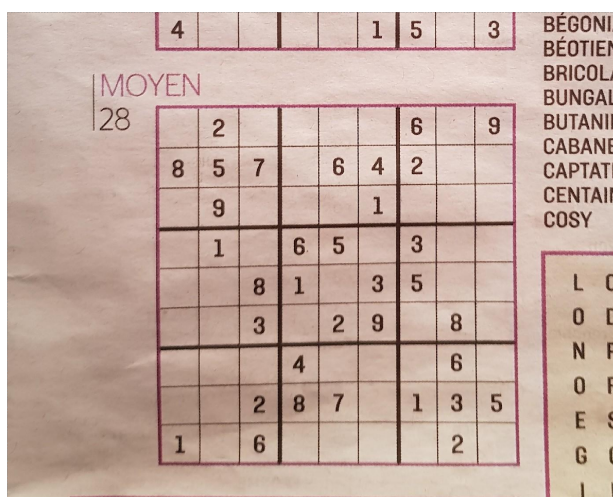


FIGURE 7 – Rotation de -45 degrés sur l'image précédemment pivotée de 45 degrés

2.4 Redimension manuelle de l'image

2.4.1 Details

Afin de faciliter la suite du traitement de l'image, nous avons implémenté un exécutable `./reformat {image width height}`, employant également les bibliothèques SDL2 et SDL_Image, et permettant de redimensionner une image en n'importe quel format *width * height*

Ce programme nous est très utile pour redimensionner une case contenant un chiffre après découpage en 28x28. En effet, notre réseau de neurones ne peut prendre en entrée que des images de ce format.

2.4.2 Résultats

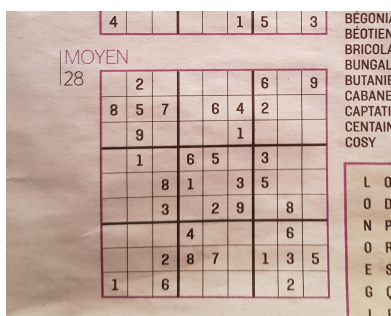


FIGURE 8 – Image originale

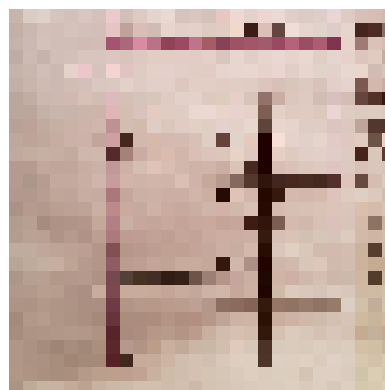


FIGURE 9 – Image redimensionnée en 28 * 28

3 Segmentation de l'image

Pour détecter la grille du Sudoku et de la segmenter afin de récupérer les 81 cases du sudoku nous avons décidé d'utiliser la transformée de Hough.

3.1 Transformée de Hough

La transformée de Hough est une technique de reconnaissance de formes. Pour notre utilisation nous utiliserons cette transformée afin de détecter l'ensemble des droites sur l'image.

3.1.1 Principe

La transformée de Hough est une méthode permettant de détecter des droites dans une image, c'est-à-dire un ensemble de points plus ou moins alignés. Cette méthode repose sur le paramétrage d'une droite par un angle θ et une distance ρ comme définis sur la figure suivante :

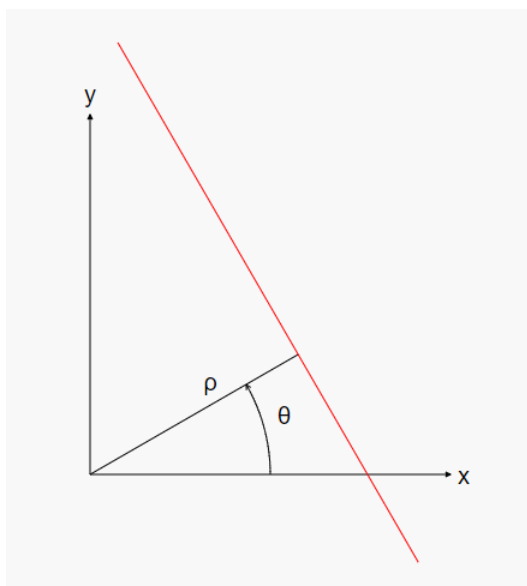


FIGURE 10 – Illustration de droite

L'équation cartésienne de la droite est :

$$\rho = x * \cos(\theta) + y * \sin(\theta)$$

L'algorithme de Hough utilise une matrice d'accumulation qui représente le plan (ρ, θ) , de dimensions (p, q) où p est le nombre de valeurs de ρ allant de $(-\text{diag}, \text{diag})$ (diag) étant la diagonale de notre image, et q le nombre de valeurs de θ allant de $(-90^\circ, 90^\circ)$. Pour chaque point (x, y) de l'image binaire traitée, chaque droite (ρ, θ) passant par ce point ajoute une unité dans l'élément correspondant de la matrice. À la fin de l'accumulation, les points de la matrice dont la valeur est la plus élevée correspondent à un grand nombre de points alignés sur l'image. Ces droites sont sélectionnées à partir d'un seuil ajustable.

Afin d'avoir de bons résultats il est important de ne pas définir un seuil unique pour chaque image. C'est pourquoi notre seuil est adaptatif en effet tout en remplissant la matrice d'accumulation en parcourant les pixels de l'image nous gardons en mémoire la valeur maximale. Une

fois la matrice d'accumulation complète et la valeur maximale trouvée nous définissons notre seuil à 50% de la valeur maximale de la matrice d'accumulation. Cela nous permet de récupérer un grand ensemble de lignes.

Pour tracer les lignes sur notre image nous parcourrons notre matrice d'accumulation et pour chaque point de coordonnées (ρ, θ) supérieures à notre seuil prédéfini précédemment, par la suite on en déduit les coordonnées cartésiennes $(x0, y0)$ à partir de la formule suivante :

$$x0 = \rho * \cos(\theta) \text{ et } y0 = \rho * \sin(\theta)$$

Avec nos coordonnées cartésiennes $(x0, y0)$ et notre diagonale (d) , on peut en déduire les coordonnées $(x1, y1)$ et $(x2, y2)$:

$$\begin{aligned} x1 &= x0 + d * \cos(\theta) \text{ et } y1 = y0 + d * \sin(\theta) \\ x2 &= x0 - d * \cos(\theta) \text{ et } y2 = y0 - d * \sin(\theta) \end{aligned}$$

Munis de toutes ces données nous pouvons tracer les droites sur notre image qui nous donne l'image suivante :

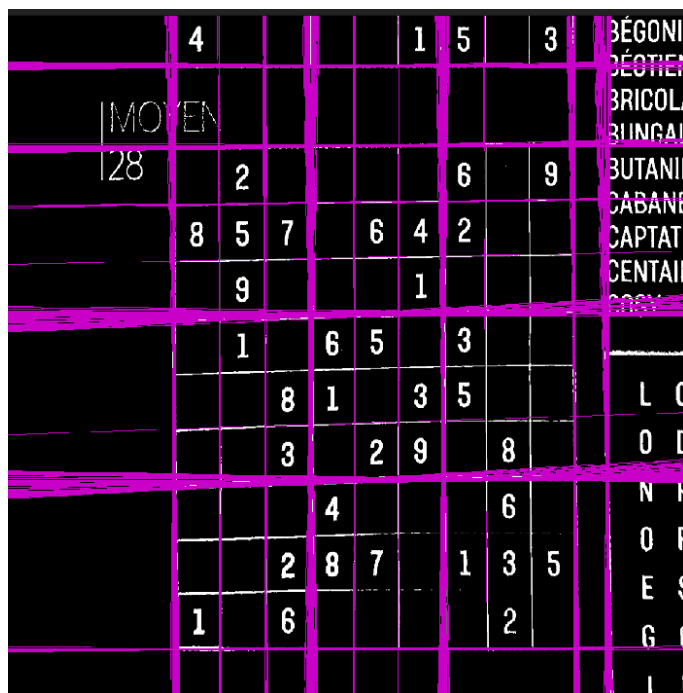


FIGURE 11 – Image avec l'ensembles des droites

Comme vous pouvez le voir nous avons détecté énormément de lignes c'est pourquoi dans une seconde étape nous allons réduire leur nombre.

Chaque ligne possède 2 points qui représente leurs extrémités. Pour chaque couple de ligne on regarde si la distance entre ces deux points est assez petite pour considérer que les 2 lignes sont assez proches pour les fusionner. Si c'est le cas nous faisons la moyenne des lignes pour en obtenir qu'une seule.

Après avoir réduit les lignes nous obtenons l'image suivante :

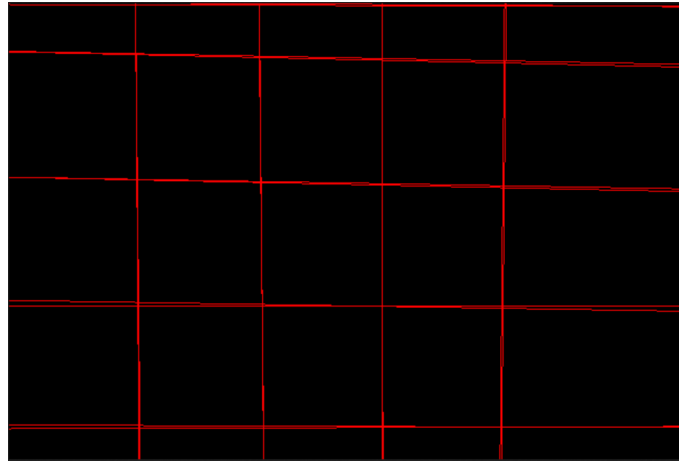


FIGURE 12 – Lignes réduites

Dans une 3e et dernières étapes nous allons déterminer le carré qui représente la grille de sudoku. Pour le détecter nous procédons comme ça :

- pour toutes les droites dans la liste de droites
- on trouve toutes les lignes avec lesquelles elles à une intersection
- on répète ce processus tant que l'on n'est pas sur la droite de référence
- on obtient un carré, on l'insère dans la liste des carrés
- on trouve le carré avec le plus grand périmètre

Ensuite nous obtenons le carré qui représente la grille de sudoku. ci-dessous une image de tout le processus :

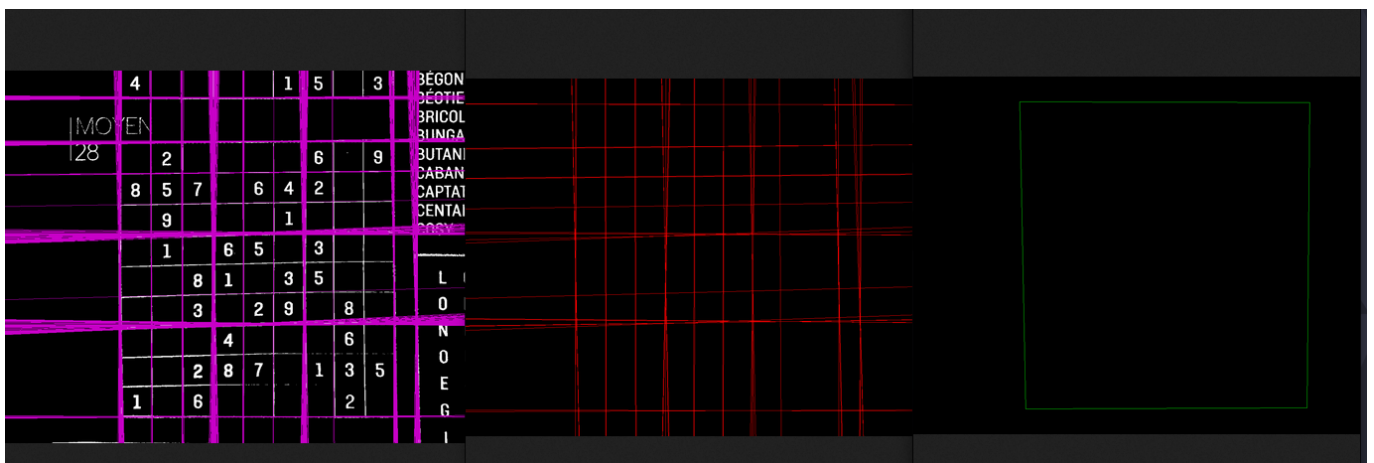


FIGURE 13 – Processus final

3.2 Découpage de l'image

Pour détecter les cases et récupérer les chiffres du sudoku, nous découpons d'abord la grille en 81 carrés de taille identique. Pour cela, nous appliquons une division par 9 de l'image sur sa hauteur et sa largeur. Cependant, cette méthode peut garder des résidus de bordures sur les images qui faussent les résultats de notre réseau de neurones. Nous appliquons donc un autre traitement sur les 81 cases de la grille pour récupérer uniquement les chiffres et de supprimer les impuretés. La méthode est la suivante : pour chaque image, notre algorithme part du centre et augmente progressivement l'image jusqu'à ne plus détecter de pixels noirs correspondant à une partie d'un chiffre. L'image du chiffre est alors redimensionnée en 28 par 28 pour notre réseau de neurones.

4	3	6	5	7	2	9	8	1
9	1	7	4	8	3	5	2	6
5	2	8	9	1	6	3	4	7
1	8	9	6	3	4	2	7	5
7	6	5	1	2	8	4	3	9
2	4	3	7	5	9	1	6	8
8	9	4	2	6	1	7	5	3
3	7	1	8	4	5	6	9	2
6	5	2	3	9	7	8	1	4

FIGURE 14 – Grille avant découpage



FIGURE 15 – case 60 avant et après traitement

4 Réseau de neurones

4.1 Fonctionnement du réseau de neurones

Nous avons choisi d'utiliser une implémentation classique pour notre réseau de neurones. Il est découpé en 2 grandes parties : une partie frontpropagation, qui permet de prédire une valeur et une partie backpropagation qui permet de corriger les erreurs et de faire évoluer le réseau.

Après avoir effectué de nombreux tests, notre réseau, permettant d'apprendre la porte XOR, est constitué de 2 noeuds d'entrée, 10 noeuds cachés et 1 noeud de sortie. Les 10 noeuds cachés nous donnent une très bonne précision et une très bonne consistance des résultats. Pour ce qui est des fonctions d'activation des noeuds, nous avons opté pour la fonction sigmoïd sur l'activation du noeud de sortie et la fonction ReLU sur l'activation des noeuds cachés. La fonction ReLU est très utile, car elle permet d'arriver à des résultats précis très rapidement.

Au final, le réseau est capable de se réinitialiser, d'apprendre la porte XOR, de prédire les résultats de la porte XOR et d'écrire les poids et biais dans un fichier.

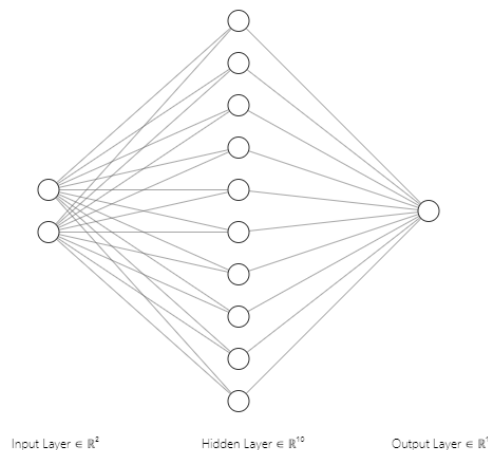


FIGURE 16 – Représentation de notre réseau de neurones

4.2 Résultats

Enfin, les résultats du réseau de neurones, pour un entraînement sur 100,000 époques avec 4 ensembles d'entraînement, sont les suivants :

```
For input [0, 0] expected 0, predicted 0 (0.008164)
For input [1, 0] expected 1, predicted 1 (0.996919)
For input [0, 1] expected 1, predicted 1 (0.996718)
For input [1, 1] expected 0, predicted 0 (0.002669)
```

FIGURE 17 – Résultats

5 Résolveur de sudoku

Notre algorithme, pour résoudre les grilles de sudoku fonctionne de manière récursive en utilisant la méthode de "backtracking". Notre résolveur est donc une méthode de brute force, qui teste toutes les possibilités jusqu'à obtenir la grille finale. Tout en respectant les consignes du cahier des charges, il fonctionne de cette façon :

- Tout d'abord, l'algorithme récupère la grille du fichier donné en paramètre.
- Pour chaque cellule vide dans la grille du sudoku, un nombre entre 1 et 9 est assigné tout en respectant les règles du sudoku.
- Si le chiffre est correct, l'algorithme passe à la case vide suivante. Sinon, si aucun chiffre en 1 et 9 ne peut être placé, l'algorithme retourne sur une case antérieure et teste pour un autre chiffre.
- Si toutes les cases du sudoku sont remplies, le programme écrit la solution dans un nouveau fichier avec le nom du fichier d'entrée suivie de l'extension ".result".

```
nico1as@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kuSolver (master)
$ ls
Makefile  grid_00  solver*  solver.c

nico1as@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kuSolver (master)
$ cat grid_00
... ..4 58.
... 721 ..3
4.3 ... ..

21. .67 ..4
.7. ... 2..
63. .49 ..1

3.6 ... ..
... 158 ..6
... ..6 95.

nico1as@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kuSolver (master)
$ ./solver grid_00

nico1as@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kuSolver (master)
$ ls
Makefile  grid_00  grid_00.result  solver*  solver.c

nico1as@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kuSolver (master)
$ cat grid_00.result
127 634 589
589 721 643
463 985 127

218 567 394
974 813 265
635 249 871

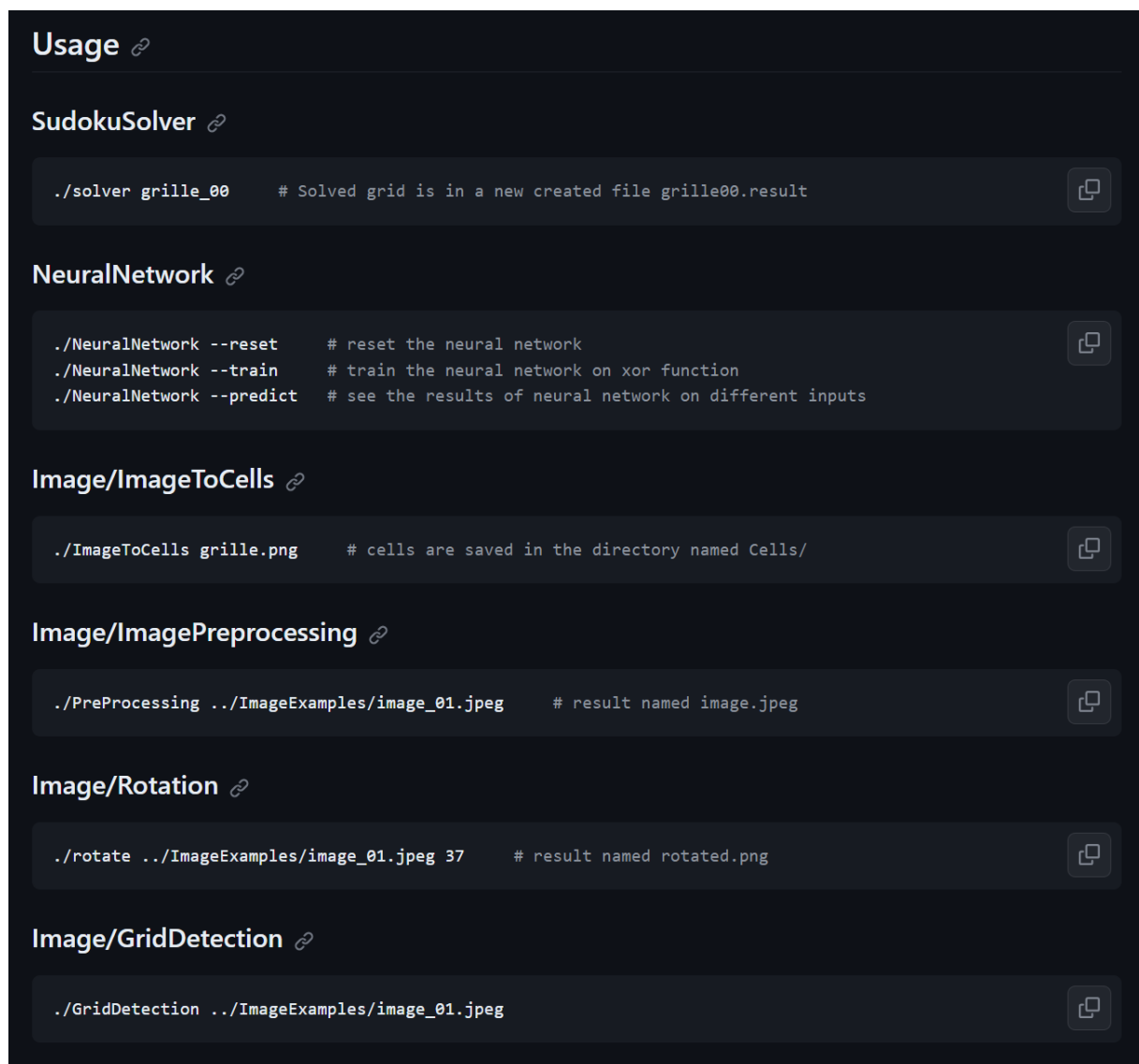
356 492 718
792 158 436
841 376 952

nico1as@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kuSolver (master)
$
```

FIGURE 18 – Utilisation du solver

6 Interface utilisateur

Pour cette première soutenance, nous n'avons pas avancé la partie interface graphique, nous avons privilégié le fonctionnement des parties plus importantes comme le réseau de neurones et la détection de la grille. L'utilisation des programmes se fait donc à l'aide du terminal. Toutes les informations pour faire fonctionner le projet se trouvent dans le README à la racine du dépôt Git.



The screenshot shows a dark-themed README file with the following sections and commands:

- Usage** (with a link icon)
- SudokuSolver** (with a link icon)
Command: `./solver grille_00` # Solved grid is in a new created file grille00.result
- NeuralNetwork** (with a link icon)
Commands:
 - `./NeuralNetwork --reset` # reset the neural network
 - `./NeuralNetwork --train` # train the neural network on xor function
 - `./NeuralNetwork --predict` # see the results of neural network on different inputs
- Image/ImageToCells** (with a link icon)
Command: `./ImageToCells grille.png` # cells are saved in the directory named Cells/
- Image/ImagePreprocessing** (with a link icon)
Command: `./PreProcessing ../ImageExamples/image_01.jpeg` # result named image.jpeg
- Image/Rotation** (with a link icon)
Command: `./rotate ../ImageExamples/image_01.jpeg 37` # result named rotated.png
- Image/GridDetection** (with a link icon)
Command: `./GridDetection ../ImageExamples/image_01.jpeg`

FIGURE 19 – Utilisation du projet

7 Conclusion

Ainsi, nous sommes globalement satisfaits du travail réalisé jusqu'à présent, malgré les quelques problèmes rencontrés en cours de route. En vue de la soutenance finale, nous visons non seulement à implémenter le reste des fonctions nécessaire à la réalisation complète du projet, mais également une interface utilisateur plus qualitative. Nous sommes pour le moment optimistes face au futur de ce projet, et espérons que nous saurons réaliser la suite du projet tout autant que ce que nous l'avons été jusqu'à présent.

Table des figures

1	Etapes du traitement	5
2	Filtre médian	6
3	Image originale	6
4	Image finale	6
5	Image originale	7
6	Rotation de 45 degrés	8
7	Rotation de -45 degrés sur l'image précédemment pivotée de 45 degrés	8
8	Image originale	9
9	Image redimensionnée en 28 * 28	9
10	Illustration de droite	10
11	Image avec l'ensembles des droites	11
12	Lignes réduites	12
13	Processus final	12
14	Grille avant découpage	13
15	case 60 avant et après traitement	13
16	Représentation de notre réseau de neurones	14
17	Résultats	14
18	Utilisation du solver	15
19	Utilisation du projet	16